



Guía n.º 1.2.4

Actividad: Métodos, clases, abstracción y encapsulamiento en Java

Análisis de una clase en Python y su implementación equivalente en Java

Guía n.º	1.2.4
Nombre de la guía	Actividad de análisis y transformación de una clase de Python a Java
Sigla	DSY1102
Asignatura	Desarrollo Orientado a Objetos
Experiencia de Aprendizaje	Reconocer clases, atributos, métodos, abstracción y encapsulamiento a partir de un ejemplo en Python y traducirlo correctamente a Java.
Tiempo	90 minutos
Modalidad de Trabajo	Individual o en parejas
Indicadores de Logro	Identifica la estructura de una clase, explica su responsabilidad, reconoce métodos y atributos, y aplica abstracción y encapsulamiento en una versión equivalente en Java.

Propósito de la guía

Desarrollar la capacidad de analizar una clase escrita en Python para reconocer su estructura orientada a objetos y luego transformarla en una clase equivalente en Java, explicando de manera clara los conceptos de clase, método, abstracción y encapsulamiento.

La guía utiliza como base una clase Auto. Primero se presenta la versión en Python, luego se describe su funcionamiento y finalmente se construye una versión equivalente en Java aplicando una organización más formal de atributos privados, constructor, getters, setters y métodos de comportamiento.

Instrucciones generales

- Lee con atención la clase base en Python antes de revisar su traducción a Java.
- Identifica la responsabilidad principal de la clase Auto.
- Observa qué atributos representan el estado del objeto y qué métodos representan su comportamiento.
- Relaciona cada elemento de la versión Python con su equivalente en Java.
- Presta especial atención a cómo se aplica el encapsulamiento en la versión Java.
- Completa los ejercicios finales utilizando nombres claros y justificaciones breves.

Formato sugerido de respuesta para cada ejercicio

1. Nombre de la clase
2. Descripción breve de la responsabilidad de la clase
3. Lista de atributos
4. Lista de métodos
5. Constructor propuesto
6. Justificación breve: ¿qué datos y acciones fueron seleccionados y por qué?



Caso base: Registro y movimiento de un automóvil

Situación: Se desea representar un automóvil dentro de un sistema simple. Del vehículo interesa guardar datos como modelo, marca, color, año, kilometraje y tipo. Además, debe poder avanzar o retroceder modificando los kilómetros registrados.

Consigna: Analiza la clase base en Python, describe sus elementos principales y luego revisa cómo se transforma a una clase equivalente en Java aplicando conceptos de programación orientada a objetos.

Clase base en Python	<pre>class Auto: # constructor def __init__(self, modelo, marca, color, año, km, tipo): self._modelo = modelo self.marca = marca self.color = color self.año = año self.km = km self.tipo = tipo # getters y setter def getModelo(self): return self._modelo def setModelo(self, nuevoModelo): self._modelo = nuevoModelo def getMarca(self): return self.marca def getColor(self): return self.color def getAño(self): return self.año def getKm(self): return self.km def getTipo(self): return self.tipo def avanzar(self, distancia): self.km = self.km + distancia print(f"Avanzando: {distancia} km. Km totales: {self.km}") def retroceder(self, distancia): self.km = self.km - distancia print(f"Retrocediendo: {distancia} km. Km totales: {self.km}")</pre>
Descripción general	La clase Auto representa un vehículo mediante atributos que almacenan su estado y métodos que permiten consultar o modificar información, además de simular su movimiento.

Descripción de la clase Auto en Python



Clase	Auto es la plantilla que define cómo serán los objetos de tipo automóvil.
Atributos	modelo, marca, color, año, km y tipo representan la información principal del vehículo.
Constructor	El método <code>__init__</code> inicializa el objeto al momento de crearlo y asigna valores a sus atributos.
Getters y setter	Permiten consultar atributos y, en el caso de modelo, modificar su valor mediante un método específico.
Método avanzar	Aumenta el kilometraje del vehículo según una distancia recibida como parámetro.
Método retroceder	Disminuye el kilometraje registrado y muestra el nuevo total de kilómetros.

Conceptos clave trabajados en la guía

Clase	Una clase es un molde o plantilla que define qué datos tendrá un objeto y qué acciones podrá realizar.
Método	Un método es una acción que la clase puede ejecutar. Puede recibir parámetros y modificar el estado del objeto.
Abstracción	La abstracción consiste en seleccionar solo las características importantes del objeto real. En este caso, del automóvil se toman solo los datos necesarios para el problema.
Encapsulamiento	El encapsulamiento organiza y protege los datos del objeto. En Java esto se refuerza declarando atributos privados y controlando el acceso mediante getters y setters.
Objeto	Un objeto es una instancia concreta de una clase. Por ejemplo, un Auto específico con una marca, modelo y kilometraje definidos.

¿Cómo se observa la abstracción en este ejemplo?

La clase Auto no intenta representar todos los detalles posibles de un vehículo real. No incluye número de puertas, tamaño de motor, historial de mantenimientos o patente, porque esos datos no son necesarios para el problema planteado. En cambio, se seleccionan solo los elementos más útiles para identificar el vehículo y simular su movimiento.

¿Cómo se observa el encapsulamiento en este ejemplo?

En la clase base de Python ya aparece la idea de encapsular el atributo modelo mediante el uso de `_modelo` y métodos de acceso. En la versión Java este concepto se aplica de forma más estricta, ya que los atributos se declaran `private` y solo pueden consultarse o modificarse a través de métodos públicos controlados.

Clase equivalente en Java

Clase en Java	<pre>public class Auto { private String modelo; private String marca; private String color; private int anio; private double km; }</pre>
----------------------	---

	<pre> private String tipo; public Auto(String modelo, String marca, String color, int anio, double km, String tipo) { this.modelo = modelo; this.marca = marca; this.color = color; this.anio = anio; this.km = km; this.tipo = tipo; } public String getModelo() { return modelo; } public void setModelo(String nuevoModelo) { this.modelo = nuevoModelo; } public String getMarca() { return marca; } public String getColor() { return color; } public int getAnio() { return anio; } public double getKm() { return km; } public String getTipo() { return tipo; } public void avanzar(double distancia) { km = km + distancia; System.out.println("Avanzando: " + distancia + " km. Km totales: " + km); } public void retroceder(double distancia) { km = km - distancia; System.out.println("Retrocediendo: " + distancia + " km. Km totales: " + km); } } </pre>
Descripción general	La versión Java mantiene la misma idea de la clase original, pero organiza sus atributos y métodos con una sintaxis más formal propia del lenguaje.

Explicación de la versión en Java



Atributos privados	Los atributos se declaran private para reforzar el encapsulamiento y evitar modificaciones directas desde fuera de la clase.
Constructor	El constructor Auto(...) recibe todos los datos necesarios para crear correctamente un objeto desde el inicio.
Getters	Los métodos get permiten consultar los valores almacenados en los atributos sin exponer directamente las variables.
Setter	El método setModelo permite cambiar el valor de modelo de manera controlada.
Métodos de comportamiento	Los métodos avanzar y retroceder representan acciones del automóvil y modifican el estado del objeto al actualizar los kilómetros.
Tipos de datos	Java exige declarar con precisión el tipo de cada atributo y parámetro, por ejemplo String, int y double.

Relación entre la clase Python y la clase Java

Elemento	En Python	En Java
Clase	Se declara con class Auto:	Se declara con public class Auto { }
Constructor	Se usa __init__	Se usa un método con el mismo nombre de la clase
Atributos	Pueden crearse al asignar self.atributo	Se declaran explícitamente al inicio de la clase
Encapsulamiento	Se sugiere con guion bajo	Se aplica con private y métodos públicos
Salida en pantalla	print(...)	System.out.println(...)
Tipos de datos	No se declaran de forma obligatoria	Se declaran de forma explícita

Ejemplo de uso de la clase en Java

Uso desde main	<pre>public class Main { public static void main(String[] args) { Auto auto1 = new Auto("Yaris", "Toyota", "Rojo", 2020, 35000, "Sedan"); System.out.println("Modelo: " + auto1.getModelo()); System.out.println("Marca: " + auto1.getMarca()); auto1.avanzar(120); auto1.retroceder(20); } }</pre>
Explicación	Primero se crea un objeto auto1 a partir de la clase Auto. Luego se consultan datos con getters y finalmente se ejecutan métodos que cambian el kilometraje del objeto.

Actividades de aplicación

Ejercicio 1: Identificación de elementos de la clase



Situación: Observa la clase Auto presentada en Python y luego en Java.

Consigna: Identifica la clase principal, sus atributos, sus métodos y explica cuál es la responsabilidad general del objeto dentro del sistema.

Clase propuesta	Auto
Atributos	modelo, marca, color, anio, km, tipo
Métodos	getters, setters, avanzar(), retroceder()
Constructor	modelo, marca, color, anio, km, tipo
Justificación	Con esta mezcla de atributos y metodos tenemos lo basico clasificar o almacenar datos de un auto. Ademas, le permitimos realizar 2 acciones (avanzar /retroceder) típicas del objeto que abstraen

Ejercicio 2: Mejora del encapsulamiento

Situación: Se desea mejorar la protección de los datos del automóvil.

Consigna: Propón una nueva versión de la clase Auto en Java en la que todos los atributos tengan sus respectivos getters y setters, justificando por qué esto fortalece el encapsulamiento.

Clase propuesta	Auto
Atributos	modelo, marca, color, anio, km, tipo
Métodos	getters, setters, avanzar(), retroceder() , @to_string
Constructor	modelo, marca, color, anio, km, tipo
Justificación	Se parte de la base de que todos los atributos estan privados y con sus respectivos getters and setters. Se anade la sobre escritura con @Override to_string para que su impresion por consola sea mas amigble

Ejercicio 3: Nuevo método de comportamiento

Situación: El sistema ahora necesita que el automóvil pueda detenerse y reportar su estado actual.



Consigna: Agrega a la clase Auto al menos un nuevo método de comportamiento y explica qué problema resuelve dentro del sistema.

Clase propuesta	Auto
Atributos	modelo, marca, color, anio, km, tipo, estado
Métodos	getters, setters, avanzar(), retroceder(), @toString(), detener()
Constructor	modelo, marca, color, anio, km, tipo
Justificación	Se crea un nuevo atributo de estado. Luego se crea la clase solicitada que cambia el valor de la variable a detenida. (Por el momento sin manejo de errores. o condicionales.

Ejercicio 4: Nueva abstracción relacionada

Situación: Además del automóvil, el sistema necesita representar a un conductor.

Consigna: Abstrae una clase Conductor. Define atributos y métodos coherentes con el contexto y explica brevemente por qué seleccionaste esos elementos.

Clase propuesta	Conductor
Atributos	nombre, edad, rut, habilitado, licencias
Métodos	getters, setters, @toString(), agregarLicencia()
Constructor	nombre, edad, rut, habilitado, licencias
Justificación	Es una clase base con la información necesaria mas un metodo toString y uno que permita agregar licencias.

Ejercicio 5: Adaptación del caso

Situación: Una empresa de transporte quiere reutilizar la idea de la clase Auto para manejar vehículos de trabajo.

Consigna: Propón una clase VehiculoEmpresa o un nombre equivalente. Mantén la idea del ejemplo base, pero adapta atributos y métodos al nuevo contexto.

Clase propuesta	VehiculoEmpresaReparto
Atributos	modelo, marca, color, anio, km, tipo, estado, enRecorrido, pedidos
Métodos	Heredados, entregaPedido(), @toString (nueva sobreescritura)
Constructor	modelo, marca, color, anio, km, tipo, estado, enRecorrido, pedidos
Justificación	Se opto por mantener todo lo anterior y solo agregar los nuevos atributos propia se un vehiculo de reparto. Ademàs, se agrega el metodo entregaPedido() para ir vacionado el almacenamiento. Y se vuelve a sobreescibir el toString para el nuevo contexto.

Criterios de evaluación sugeridos

Categoría	% logro	Descripción de niveles de logro
Comprensión de la clase base	25%	Reconoce correctamente la estructura de la clase y explica su responsabilidad dentro del problema.
Identificación de atributos y métodos	25%	Distingue con claridad los datos y comportamientos principales del objeto.
Aplicación de abstracción y encapsulamiento	25%	Selecciona elementos pertinentes y justifica adecuadamente cómo se organizan y protegen los datos.
Claridad de la traducción a Java	25%	Presenta una versión coherente, ordenada y comprensible de la clase en Java.

Cierre

Al finalizar esta guía, el estudiante debería ser capaz de analizar una clase sencilla en Python, identificar sus componentes principales y transformarla en una clase equivalente en Java, comprendiendo cómo intervienen los conceptos de clase, métodos, abstracción y encapsulamiento en el diseño orientado a objetos.